The Connected Digital Twin

Group Assignment 1 | Course: Digital Twins

Project 1: The Connected Digital Twin (Foundational)

Ian Thomas Mathew 1010766
Emmanuel Kevin Suraiskumar 1011048
Suhani Kalra 1010844
Gunhyun Jeong 1010966

Connected Digital Twin of a Virtual Smart Room

1. Introduction

1.1 Background

A Digital Twin is a digital representation of a physical asset, system, or environment that is continuously refreshed using data from sensors or data streams. Unlike a static digital model, a Digital Twin reflects the current state of its physical counterpart through continuous data synchronization. This flow of information from the asset to the digital system is known as the Digital Thread.

This project implements a Connected Digital Twin of a virtual SUTD lab room using simulated IoT sensor data. The system monitors three environmental parameters: temperature, humidity, and occupancy. These virtual sensors generate synthetic readings every five seconds, which are transmitted using MQTT, stored in InfluxDB, and visualized through a real-time Grafana dashboard.

The implementation also maintains an explicit digital twin state. Instead of showing raw charts only, the system derives comfort status, occupancy level, active alert status, and alert messages from the latest sensor readings. This makes the dashboard a digital-twin interface rather than only a telemetry viewer.

The main objective is to demonstrate the foundational components of a Connected Digital Twin: asset virtualization, synthetic data ingestion, connectivity, time-series storage, dashboard visualization, and threshold-based alerting. Although physical sensors are not used, the system follows a realistic IoT architecture that can be extended to real-world smart building deployments.

2. System Overview

The system consists of seven integrated components. These components make the prototype reproducible and explicit as a Connected Digital Twin: an asset model, derived twin-state computation, and a 3D room model embedded in Grafana.

Component	Role in the Connected Digital Twin	Implementation evidence
Virtual SUTD Lab Room and Asset Model	Represents the monitored room, its zones, sensor locations, capacity, units, and thresholds.	config/ lab_asset_model.json
Python Sensor Simulator	Generates synthetic temperature, humidity, and occupancy readings every five seconds.	simulator/ sensor_simulator.py
Eclipse Mosquitto MQTT Broker	Transports sensor messages using MQTT publish-subscribe topics.	lab/sensors/temperature, lab/sensors/humidity, lab/sensors/occupancy
Python MQTT-to-InfluxDB Bridge	Subscribes to MQTT topics, stores raw readings, derives room state, and records alerts.	simulator/mqtt_bridge.py
InfluxDB v2	Stores timestamped raw readings, derived twin-state records, and alert-event evidence.	bucket: sensors
Grafana Dashboard	Displays current state, sensor streams, alert history, and the embedded 3D model.	SUTD Lab Connected Digital Twin
Nginx 3D Model Service	Serves the local 3D lab-room model for the Grafana iframe panel.	http://localhost:8050

The Python simulator publishes each sensor reading as a JSON payload to MQTT topics such as:

```
lab/sensors/temperature
lab/sensors/humidity
lab/sensors/occupancy
```

The MQTT bridge subscribes to lab/sensors/# so that all current and future sensor topics under the same hierarchy can be consumed without changing the subscription pattern.

3. System Architecture

The system is designed around the six-layer Digital Twin architecture. The implementation maps each layer to a working technical component and keeps the data flow traceable from the virtual asset to dashboard evidence.

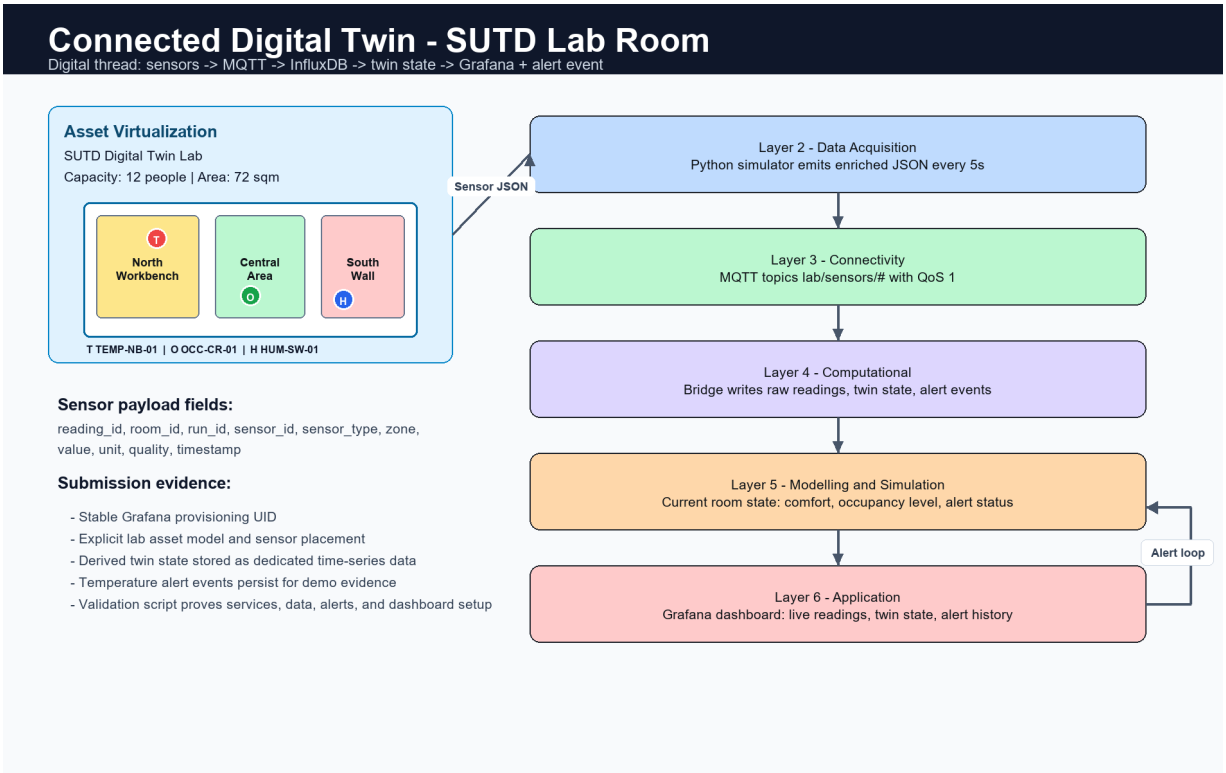

Figure 1. Six-layer architecture, digital thread, room layout, and alert loop.

3.1 Physical Layer

The Physical Layer represents the virtual SUTD lab room and its environmental sensors. The sensors are software-based virtual sensors, but they are modeled as if they were placed in specific areas of a teaching lab.

The asset model defines the monitored room as sutd-lab-room-1, a 72 square metre teaching lab with a capacity of 12 people. The model includes three room zones and maps each virtual sensor to a zone and normalized room position.

Sensor ID	Sensor Type	Zone	Unit	Purpose
TEMP-NB-01	temperature	north_bench	degC	Temperature probe near heat-generating lab equipment
HUM-SW-01	humidity	south_wall	%	Humidity sensor on the lower wall away from equipment
OCC-CR-01	occupancy	central_room	people	Occupancy counter covering the collaboration area

The temperature sensor includes normal variation and occasional spikes above 30 degC to test the dashboard alert path. Humidity follows a smooth fluctuating pattern with random noise, while occupancy changes dynamically to simulate people entering and leaving the lab.

3.2 Data Acquisition Layer

The Data Acquisition Layer is implemented using a Python sensor simulator. The simulator generates synthetic readings every five seconds. Each reading is converted into a JSON payload before being published to the MQTT broker.

Each payload includes sensor identity, room identity, zone, quality, run ID, and timestamp. This makes each reading traceable and suitable for both raw storage and twin-state computation.


```
{
  "reading_id": "TEMP-NB-01-000001",
  "room_id": "sutd-lab-room-1",
  "room_name": "SUTD Digital Twin Lab",
  "run_id": "manual",
  "sensor_id": "TEMP-NB-01",
  "sensor": "temperature",
  "sensor_type": "temperature",
  "zone": "north_bench",
  "value": 28.4,
  "unit": "degC",
  "quality": "GOOD",
  "timestamp": 1720861200.45
}
```


Each sensor is published to its own MQTT topic. This makes the data stream modular and allows each sensor type to be processed or visualized independently.

3.3 Connectivity Layer

The Connectivity Layer uses MQTT through an Eclipse Mosquitto broker. MQTT follows a publish-subscribe model, where the sensor simulator acts as the publisher and the MQTT-to-InfluxDB bridge acts as the subscriber.

The publisher sends messages to specific sensor topics, while the subscriber listens to the wildcard topic lab/sensors/#. This allows the bridge to receive temperature, humidity, and occupancy readings without subscribing to each topic individually.

MQTT was selected because it is lightweight, event-driven, and well-suited for frequent IoT telemetry. Unlike HTTP, which usually requires repeated request-response polling, MQTT allows new sensor readings to be pushed immediately to subscribers. This reduces communication overhead and supports near real-time synchronization between the virtual room and its digital representation.

The simulator publishes messages using QoS 1, which provides at-least-once delivery. This improves reliability by ensuring that sensor readings are delivered to the broker even if occasional network issues occur.

3.4 Computational Layer

The bridge writes three evidence streams to InfluxDB. This separates raw telemetry from interpreted twin-state records and alert-event history, which makes the data model easier to explain and easier to validate.

InfluxDB measurement	Purpose	Key tags or fields
lab_sensor_readings	Raw time-series telemetry from each virtual sensor.	room_id, run_id, sensor_id, sensor_type, zone, quality, value, unit
lab_twin_state	Derived digital twin state after temperature, humidity, and occupancy have been received.	temperature, humidity, occupancy, comfort_status, occupancy_level, alert_active, alert_message, temperature_threshold
lab_alert_events	Historical evidence when the temperature crosses the alert threshold.	event_type, severity, sensor_id, zone, value, threshold, message, active

InfluxDB acts as the time-series storage layer. It organizes incoming records by time, sensor metadata, room context, and derived state, allowing Grafana to query both current and historical values efficiently.

3.5 Modelling and Simulation Layer

The Modelling and Simulation Layer is represented by two linked parts: the lab asset model and the twin-state rules. The asset model defines room metadata, zones, sensors, units, and thresholds. The twin-state rules derive comfort status, occupancy level, and alert status from the latest complete set of readings.

Comfort status is classified as OVERHEAT, WARM, HUMID, COOL, or COMFORTABLE. Occupancy level is classified as EMPTY, LIGHT, ACTIVE, or CROWDED. When the temperature is above 30 degC, alert_active becomes true and the bridge writes a new lab_alert_events record.

The model is still appropriate for a Connected Digital Twin because the main goal is real-time synchronization and monitoring rather than autonomous control. The architecture also provides a foundation for future intelligent modelling, such as anomaly detection, occupancy forecasting, or automatic HVAC response.

3.6 Application Layer

The Application Layer is implemented using a provisioned Grafana dashboard titled SUTD Lab Connected Digital Twin. The dashboard uses a stable InfluxDB datasource UID, queries the current sensors bucket, refreshes every five seconds, and includes both live telemetry and derived digital twin state.

Dashboard panel	Purpose
3D Lab Room Model	Shows the virtual room, zones, and sensor placement through an embedded local 3D model.
Current Temperature	Shows the latest derived temperature from lab_twin_state.
Temperature Alert State	Shows whether the current twin state is above the 30 degC threshold.

Current Occupancy	Shows the latest occupancy value from the twin state.
Twin State Message	Displays the current alert message and related state fields.
Temperature Stream with 30 degC Threshold	Shows live temperature history and the configured threshold line.
Humidity Stream	Shows live humidity history over the selected time window.
Occupancy Stream	Shows live occupancy changes over time.
Temperature Alert Event History	Lists historical threshold events from lab_alert_events.

4. Digital Thread and Data Flow

The Digital Thread refers to the continuous and traceable flow of data from the virtual sensors to the dashboard. In this project, the Digital Thread is established through the following pipeline:

```
Virtual SUTD Lab Room and Asset Model
  -> Python Sensor Simulator
  -> Enriched JSON Sensor Payloads
  -> Eclipse Mosquitto MQTT Broker
  -> Python MQTT-to-InfluxDB Bridge
  -> InfluxDB raw readings + twin state + alert events
  -> Grafana dashboard with 3D model, live panels, and alert history
  -> User monitoring and demonstration evidence
```

Each sensor reading is generated by the Python simulator, packaged as an enriched JSON message, and published to the MQTT broker. The bridge receives the message, writes the raw reading into InfluxDB, updates the latest readings cache, derives the room state when all three sensor types are available, and writes alert events when the temperature threshold is breached.

This creates a stronger digital thread because every layer has evidence: MQTT topics show transport, InfluxDB measurements show persistence, twin-state records show interpretation, alert-event records show the warning loop, and Grafana panels show the final user-facing view.

5. Technology Selection

Component	Selected technology	Reason
Programming language	Python	Simple implementation, strong IoT library support, and readable simulator logic.
Sensor data format	JSON	Lightweight, human-readable, and easy to enrich with room, sensor, zone, quality, and timestamp metadata.

Messaging protocol	MQTT	Lightweight publish-subscribe protocol suitable for continuous IoT telemetry.
MQTT broker	Eclipse Mosquitto	Open-source broker suitable for local deployment and topic-based routing.
Data storage	InfluxDB v2	Optimized for timestamped sensor data, derived state, and event records.
Dashboard	Grafana 13.1.0	Supports live charts, stat panels, table panels, provisioning, InfluxDB integration, and alert visualization.
3D model service	Nginx	Serves the local self-contained 3D lab-room model embedded in Grafana.
Deployment support	Docker Compose	Runs Mosquitto, InfluxDB, Grafana, and the 3D model service reproducibly.

This stack was selected because it is lightweight, open-source, and representative of real IoT-based Digital Twin systems. Each component has a clear role in the architecture, making the system modular and easy to extend.

6. Messaging Protocol Justification

MQTT was selected as the messaging protocol because it is designed for lightweight, frequent data transmission in IoT systems. It uses a publish-subscribe model, where publishers and subscribers do not communicate directly. Instead, all communication passes through a broker.

This design is useful for Digital Twin applications because multiple systems can consume the same sensor feed. For example, the current system sends data to InfluxDB, but future versions could add another subscriber for anomaly detection or automated control without changing the sensor simulator.

Compared with HTTP, MQTT is more suitable for this project because HTTP typically relies on request-response communication. For continuous sensor streams, this would require frequent polling, which introduces unnecessary overhead. MQTT allows readings to be pushed as soon as they are generated.

MQTT	**HTTP**
Publish-subscribe model	Request-response model
Event-driven updates	Often requires polling
Lightweight message overhead	Higher overhead for frequent telemetry
Suitable for continuous IoT streams	Better suited for occasional web/API requests
Supports topic hierarchy and QoS	Reliability is usually handled at the application layer

The project uses QoS 1, meaning each message is delivered at least once. This is useful for maintaining an accurate Digital Twin state because it reduces the chance of missing sensor updates. The trade-off is that duplicate messages are possible, but this is acceptable for a foundational prototype.

7. Dashboard and Alerting

The Grafana dashboard is the main user-facing interface of the Connected Digital Twin. It is available locally at http://localhost:3000 and is provisioned as SUTD Lab Connected Digital Twin. The dashboard combines raw sensor streams, derived twin-state indicators, an alert history table, and an embedded 3D lab-room model.

The top section gives an immediate operational view: the 3D Lab Room Model shows the virtual room and sensor placement, Current Temperature shows the latest temperature, Temperature Alert State shows whether the 30 degC threshold is active, Current Occupancy shows people count, and Twin State Message explains the current condition in words.

The middle and lower sections provide historical evidence. The Temperature Stream includes a 30 degC threshold line, the Humidity Stream and Occupancy Stream show live environmental trends, and Temperature Alert Event History lists recorded events from lab_alert_events. This means the warning is not only a visual line on a chart; it is also stored as queryable event evidence.

The embedded 3D model is served separately at http://localhost:8050 and appears inside Grafana through an iframe panel. This makes the dashboard more understandable during demonstration because viewers can connect the time-series readings to the room zones and sensor placement.

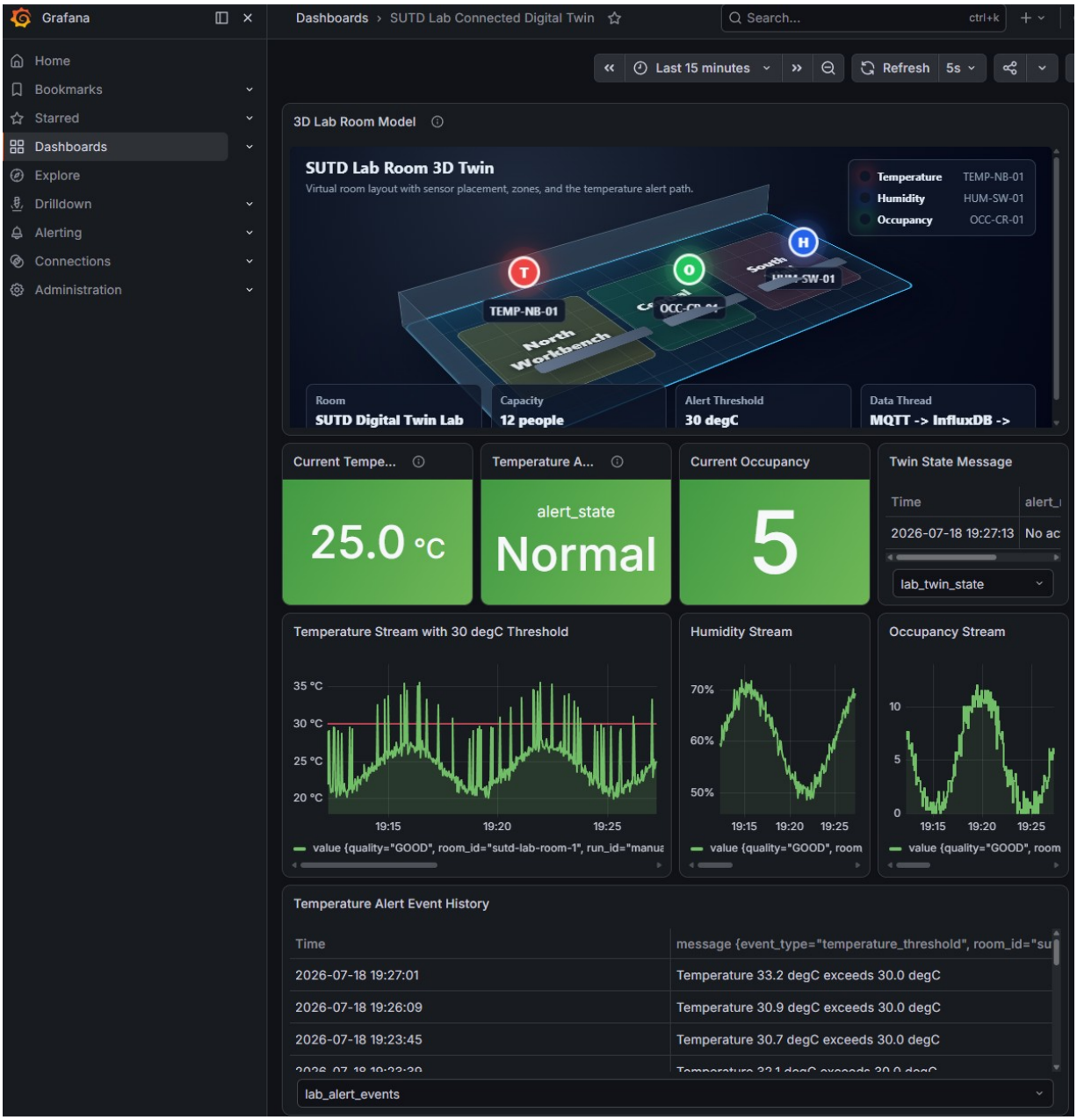

Figure 2. Grafana dashboard with embedded 3D lab-room model, current state panels, live streams, and temperature alert event history.

8. Results and Discussion

The prototype successfully demonstrates a Connected Digital Twin of a virtual SUTD lab room. The Python simulator generated temperature, humidity, and occupancy readings every five seconds and published them to the Mosquitto MQTT broker. The MQTT-to-InfluxDB bridge successfully subscribed to the lab/sensors/# topic pattern and stored incoming readings in InfluxDB.

Grafana successfully queried both raw and derived records. The current-state panels used lab_twin_state, the trend panels used lab_sensor_readings, and the alert history panel used lab_alert_events. This confirms that the dashboard is connected to the full digital thread rather than only plotting raw values.

The 30 degC alert scenario was validated by forcing a high-temperature simulator run. When the temperature exceeded the threshold, the bridge derived an OVERHEAT state, set alert_active to true, and wrote a lab_alert_events record with severity, sensor ID, zone, measured value, threshold, and message.

The implementation also includes a repeatable validation command: python scripts/validate_demo.py. This command checks that Mosquitto, InfluxDB, Grafana, and the 3D model service are reachable; verifies Python dependencies; runs a short forced-alert scenario; queries InfluxDB for fresh raw readings, twin state, and alert events; and checks that Grafana provisioning uses the stable datasource UID.

The system also demonstrates modularity. Additional sensors can be incorporated by publishing to additional MQTT topics, and other applications can subscribe to the same broker without modifying the existing simulator. This makes the architecture scalable for future extensions.

8.1 Assignment Requirement Coverage

The implementation addresses the stated Project 1 requirements through the following report and prototype evidence.

Requirement	Implementation evidence	Where shown
Asset virtualization	Virtual SUTD lab room, room zones, sensor IDs, units, placement, and thresholds.	Sections 2, 3.1, Figure 1
Synthetic data ingestion	Python simulator publishes enriched JSON telemetry over MQTT topics.	Sections 3.2, 3.3
Architecture mapping	Six-layer architecture diagram traces sensors, data flow, twin state, dashboard, and alert loop.	Section 3, Figure 1
Dashboard development	Grafana dashboard shows live readings, current twin state, 3D room model, and 30 degC alert history.	Section 7, Figure 2
Working prototype	Docker Compose runs Mosquitto, InfluxDB, Grafana, and the 3D model service.	Sections 2, 5, 8
Technical summary	MQTT justification and data storage model explain why the stack suits a connected digital space.	Sections 5, 6

9. Limitations and Future Work

The current prototype remains a Connected Digital Twin focused on real-time monitoring. It does not yet perform predictive analytics, automated control, or physical actuation, so it should not be described as an Intelligent or Autonomous Digital Twin.

9.1 Current Limitations

- The system uses simulated data rather than physical IoT sensors.
- Only three raw environmental variables are simulated: temperature, humidity, and occupancy. The system also derives additional twin-state information such as comfort status, occupancy level, and alert state.
- The Digital Twin is mainly unidirectional; data flows from sensors to the dashboard, but no control signal is sent back to actuators.
- Alerting is threshold-based and does not use machine learning.
- The Modelling and Simulation Layer computes rule-based state, but it does not yet perform predictive or physics-based simulation.
- The system is deployed locally rather than on a cloud IoT platform.
- Security features such as MQTT authentication, TLS encryption, and role-based access control are not fully implemented.

9.2 Future Improvements

- Physical sensors such as ESP32 or Raspberry Pi-based temperature, humidity, and occupancy sensors can replace the Python simulator.
- The architecture can be deployed to AWS IoT Core, Azure IoT Hub, or Google Cloud to improve remote access and scalability.
- Machine learning models can be incorporated for anomaly detection and forecasting using historical temperature and occupancy patterns.
- The system can be made bidirectional by adding actuator commands, such as automatically requesting cooling when temperature exceeds 30 degC.
- Security can be improved through MQTT authentication, TLS encryption, stronger Grafana access control, and careful secret management.

10. Conclusion

This project successfully implements a Connected Digital Twin of a virtual SUTD lab room using simulated IoT sensor data. The system demonstrates a complete data pipeline from virtual sensor generation to real-time dashboard visualization. Python is used for sensor simulation and MQTT-to-InfluxDB bridging, Mosquitto provides lightweight MQTT communication, InfluxDB stores time-series records, and Grafana displays the live dashboard.

The prototype strengthens the assignment evidence through an explicit asset model, enriched MQTT payloads, derived lab_twin_state records, stored lab_alert_events, a provisioned Grafana dashboard, an

embedded 3D model, and a repeatable validation loop. These elements make the Digital Thread more visible and make the demonstration easier to reproduce.

The implementation satisfies the key assignment requirements: asset virtualization, synthetic data ingestion, architecture mapping, dashboard development, alerting, validation, and technical justification of the messaging protocol. Although the current prototype uses simulated data and unidirectional communication, it provides a strong foundation for future development toward a more intelligent and autonomous Digital Twin.

References

1. SUTD Journey of the Digital Twins Course. (2026). Project 1: The Connected Digital Twin (Foundational) [Assignment brief].
2. Digital Twin Consortium. (n.d.). The definition of a digital twin. Retrieved July 18, 2026, from https://www.digitaltwinconsortium.org/initiatives/the-definition-of-a-digital-twin/
3. OASIS Open. (2019, March 7). MQTT Version 5.0. OASIS Standard. Retrieved July 18, 2026, from https://www.oasis-open.org/standard/mqtt-v5-0-os/
4. Eclipse Foundation. (n.d.). Eclipse Mosquitto: An open source MQTT broker. Retrieved July 18, 2026, from https://mosquitto.org/
5. Eclipse Foundation. (n.d.). Eclipse Paho MQTT Python Client documentation. Retrieved July 18, 2026, from https://eclipse.dev/paho/files/paho.mqtt.python/html/
6. InfluxData. (n.d.). InfluxDB OSS v2 documentation. Retrieved July 18, 2026, from https://docs.influxdata.com/influxdb/v2/
7. Grafana Labs. (n.d.). Grafana OSS and Enterprise documentation. Retrieved July 18, 2026, from https://grafana.com/docs/grafana/latest/
8. Docker Inc. (n.d.). Docker Compose documentation. Retrieved July 18, 2026, from https://docs.docker.com/compose/